

Chapter 11: Short Report / Description of the Project

11.1 Project Title

Project Name: LibraAI – AI-Based Institutional Library Management System

Languages / Technologies Used:

1. Frontend Tier: React 19, TypeScript 5.7, Vite 6.2, Tailwind CSS 3.4, Lucide React icons, Recharts 2.15 data visualization suite, Zustand state management, Axios HTTP client, HTML5-QRCode optical scanner library.
2. Backend Tier: Node.js runtime environment, Express 4.21 web framework, TypeScript 5.7, Prisma ORM 6.4, Zod schema validation, JSON Web Tokens (jsonwebtoken 9.0), bcryptjs password hashing, Multer multipart handler, xlsx spreadsheet parser, pdf-parse text extraction engine.
3. Database Infrastructure: PostgreSQL 15 relational database hosted on Supabase Cloud, featuring PgBouncer connection pooling, foreign key constraints, composite unique indexes, and B-tree index optimizations.
4. AI & Intelligence Services: OpenRouter API integrating Google Gemini 2.5 Flash / Flash Lite Large Language Models, custom TF-IDF semantic vector similarity matching, and SHA-256 prompt response caching (AICache).

11.2 Project Overview

- **Objective:** To modernize and centralize higher education library operations by replacing error-prone manual ledgers and isolated spreadsheets with an automated, role-based cloud platform integrated with conversational AI study assistants and real-time business intelligence analytics.
- **System Architecture:** A decoupled three-tier client-server architecture. The presentation layer (React SPA) dispatches authenticated HTTPS/JSON requests with Bearer JWT headers to the Express application server, which executes business logic and interfaces with a Supabase-managed PostgreSQL database via Prisma ORM.
- **Key Features:** Granular 4-tier Role-Based Access Control (Student, Faculty, Librarian, Administrator); an institutional catalog holding 2,299 normalized titles and 8,246 physical copies imported via Excel; barcode-driven rapid circulation (issue, return, renewal, reservation); automated overdue fine calculation (₹5/day); interactive 22-chart Recharts analytics dashboard; LibraBot LLM study assistant; syllabus-to-book matcher; real-time reading hall seat reservation; and cryptographic SHA-256 activity audit logging.
- **Functional Modules:** Guest & Student Discovery Portal, Faculty Curriculum Shelf, Librarian Circulation Desk, Administrative Control Hub, AI Semantic Services & Study Assistant, and Executive Analytics & Reporting Module.

11.3 Problem Statement

Academic institutions frequently struggle with outdated, manual library administration systems or fragmented spreadsheet trackers. In such legacy environments, locating physical textbooks across vast campus collections requires laborious physical searches, while manual record-keeping leads to unrecorded loans, misplaced inventory, and overlooked overdue fines. Library administrators lack real-time visibility into collection turnover, seat occupancy, and department-wise demand trends. Furthermore, students receive no intelligent study support, personalized reading recommendations, or rapid syllabus alignment. A centralized, cloud-hosted, and AI-enabled library management system is required to eliminate operational inefficiencies, enforce strict audit trails, and elevate academic research capabilities.

11.4 Existing System vs Proposed System

Aspect	Existing (Manual / Spreadsheet) Approach	Proposed System (LibraAI)
Catalog Discovery	Paper index cards or static Excel files; slow manual keyword lookups.	Instant multi-attribute & semantic vector search across 8,246 physical copies.
Circulation Processing	Handwritten ledger entries prone to human transcription errors.	Barcode-driven rapid circulation with automated status and inventory validation.
Due Dates & Fines	Manual date calculation; overdue returns frequently missed.	Automated 14-day tracking with programmatic fine calculation (₹5/day).
Reports & Analytics	Tedious end-of-semester tallying; no graphical trend charts.	Live 22-chart Recharts dashboard showing footfall and category distributions.
Security & RBAC	Shared spreadsheets with zero access restrictions or audit logs.	Secure JWT tokens, bcrypt password hashing, and 4-tier granular RBAC.
Accessibility	Restricted strictly to on-premise physical library hours.	24/7 web access across desktop and mobile devices via cloud PostgreSQL.
Student Assistance	Dependent on staff availability; no study guidance tools.	24/7 LibraBot AI chatbot, syllabus matcher, and PDF study summaries.
Facility Management	Unmonitored reading halls; no seat reservation records.	Interactive digital seat booking with live zone occupancy tracking.

Table 11.1 - Comparison of systems

11.5 System Architecture

LibraAI is built upon a layered three-tier architectural paradigm designed for scalability, security, and high maintainability. The client-side Single Page Application (SPA) dispatches asynchronous HTTPS requests carrying JSON payloads and Bearer JWT authorization tokens to the Express application tier. The backend router layer enforces strict authentication and role-based authorization filters before delegating requests to controller services. Data access operations are executed through Prisma ORM, which generates type-safe SQL queries against the cloud PostgreSQL database. For intelligent services, the backend dispatches contextual prompts to OpenRouter LLM endpoints, utilizing SHA-256 database caching to minimize latency and API overhead.

Architecture Layer	Primary Responsibilities	Technologies Used
Presentation Layer	User interfaces, responsive Recharts visualization, barcode scanning, client routing.	React 19, TypeScript, Vite 6, Tailwind CSS, Axios
Application Tier	RESTful routing, JWT authentication, RBAC middleware, business logic, fine engine.	Node.js, Express 4.21, TypeScript, Zod, bcryptjs
Data Access Layer	Type-safe database abstraction, connection pooling, schema migrations.	Prisma ORM 6.4 Client & Query Engine
Data Storage Tier	Relational persistence, ACID transactions, foreign keys, B-tree indexes.	PostgreSQL 15 (Supabase Cloud Database)
AI & Intelligence	Conversational Q&A, semantic search, syllabus matching, Ask-the-Book RAG.	OpenRouter API (Gemini 2.5 Flash), SHA-256 AICache

Table 11.2 - Three-tier breakdown

11.6 Module Description

- **Authentication & Access Control Module:** Manages user registration, credential verification, bcrypt password hashing, and JWT session lifecycle. Enforces 4-tier Role-Based Access Control (Student, Faculty, Librarian, Administrator) across all protected frontend routes and backend REST endpoints.
- **Catalog & Inventory Management Module:** Maintains the institutional repository of 2,299 normalized titles and 8,246 physical copies with unique barcodes (e.g. D-1 to D-8246) and physical rack locators (A1–C4). Supports bulk Excel inventory ingestion, multi-attribute filtering, and real-time copy availability tracking.

- **Circulation & Fine Management Module:** Handles automated checkout, return, renewal, and reservation workflows. Automatically computes 14-day due dates, tracks overdue loans, applies programmatic penalties at ₹5/day upon return validation, and supports librarian fee waivers with audit logs.
- **AI Study Assistant & Semantic Search Module:** Integrates Google Gemini 2.5 Flash via OpenRouter for conversational academic Q&A, TF-IDF semantic book search, automated syllabus-to-textbook matching, and PDF study summary generation with SHA-256 response caching.
- **Analytics & Business Intelligence Module:** Aggregates transactional database records into 22 interactive Recharts visualizations, displaying 30-day footfall curves, category holding donuts, top borrowed titles, peak hourly traffic heatmaps, and reading hall seat occupancy.
- **Administrative Governance & Cryptographic Audit Module:** Provides administrators with account provisioning tools, privilege assignment, 99.98% SLA telemetry monitoring, and a tamper-evident SHA-256 hash-chained activity ledger recording every critical system action.

11.7 Database Design

The relational database schema is normalized to Third Normal Form (3NF) to eliminate data redundancy and preserve referential integrity. Deployed on Supabase PostgreSQL, it features foreign key cascading, unique constraints, and B-tree indexes on frequently queried search attributes.

Table Name	Important Columns & Constraints	Functional Purpose
User	id (PK), memberId (UQ), email (UQ), passwordHash, role, department, status	Stores member credentials and 4-tier RBAC authorization flags.
Book	id (PK), title, author, isbn, department, resourceType, embedding	Maintains normalized academic title records and vector embeddings.
BookCopy	id (PK), bookId (FK), barcode (UQ), location, shelfNumber, status	Tracks physical copy inventory mapped to library rack coordinates.
IssueRecord	id (PK), bookCopyId (FK), userId (FK), issuedDate, dueDate, status	Records active loans, return timestamps, and renewal counters.
Fine	id (PK), issueRecordId (FK), userId (FK), amount, daysOverdue, status	Audits overdue financial penalties assessed during book return.
Reservation	id (PK), bookId (FK), userId (FK), queuePosition, status	Manages patron book hold queues with priority scoring.
SeatBooking	id (PK), userId (FK), seatCode, roomType, startTime, endTime, status	Tracks reading hall and digital lab reservations in real time.
ActivityLog	id (PK), userId (FK), action, details, currentHash, previousHash	Provides a tamper-evident SHA-256 cryptographic audit trail.

Table 11.3 - Database tables

```
CREATE TABLE "IssueRecord" (
  "id" TEXT PRIMARY KEY DEFAULT gen_random_uuid(),
  "bookCopyId" TEXT NOT NULL REFERENCES "BookCopy"("id") ON DELETE RESTRICT,
  "userId" TEXT NOT NULL REFERENCES "User"("id") ON DELETE CASCADE,
  "issuedDate" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "dueDate" TIMESTAMP(3) NOT NULL,
  "returnedDate" TIMESTAMP(3),
  "renewCount" INTEGER NOT NULL DEFAULT 0,
  "status" TEXT NOT NULL DEFAULT 'ACTIVE',
  "issuedBy" TEXT,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Listing 11.1 - IssueRecord table definition

Key Entity Relationships: User (1:N) IssueRecord, Book (1:N) BookCopy, BookCopy (1:N) IssueRecord, IssueRecord (1:N) Fine, User (1:N) Reservation, User (1:N) SeatBooking.

11.8 REST API Endpoints & Integration

The Express backend exposes a comprehensive suite of RESTful endpoints adhering to standard HTTP verbs and JSON serialization. Every endpoint is protected by JWT authentication middleware and validated against declarative Zod schemas.

API Endpoint & Method	Access Authorization	Functional Description
POST /api/auth/login	Public	Authenticates member credentials and issues signed JWT bearer tokens.
GET /api/auth/me	All Roles	Fetches authenticated profile, active loans count, and reading streak metrics.
GET /api/books	Public	Retrieves paginated catalog titles with multi-filter and keyword parameters.
POST /api/books/import-excel	Admin	Parses multipart Excel workbook to bulk insert titles and physical copies.
POST /api/issues/issue	Librarian / Admin	Executes book checkout, verifies eligibility, and assigns 14-day due date.
POST /api/issues/return	Librarian / Admin	Validates book return, updates copy status, and assesses ₹5/day fine if overdue.
POST /api/ai/chat	Authenticated	Processes conversational queries with catalog-aware prompt context.
POST /api/ai/semantic-search	Public	Performs hybrid vector similarity and keyword search across book records.
GET /api/analytics/comprehensive	Admin / Librarian	Generates consolidated data series for 22 Recharts visualization components.

Table 11.4 - Core REST API endpoints

11.9 Security, Auditability & Performance Optimization

- **Authentication & Password Hashing:** User credentials are protected using bcryptjs with 10 salt rounds. Client sessions rely on HMAC-SHA256 signed JWT tokens containing member ID, expiration timestamps, and authorized role claims.
- **Cryptographic Audit Ledger:** Sensitive transactions (book checkouts, catalog imports, fee waivers) append immutable records to the ActivityLog table using SHA-256 hash chaining (currentHash = SHA256(previousHash + userId + action + timestamp)), ensuring complete non-repudiation.
- **Database Connection Pooling:** Backend services utilize PgBouncer on Supabase Cloud to maintain persistent pool connections, eliminating connection starvation during concurrent query bursts.
- **AI Inference Caching:** Prompt-response pairs are cached in the AICache table using SHA-256 prompt hashing with a 24-hour TTL, slashing redundant API calls and maintaining instantaneous sub-100ms response times.

11.10 Project Conclusion

LibraAI successfully achieves its primary objective of modernizing institutional library management through a robust, cloud-native web architecture combined with state-of-the-art generative artificial intelligence. By automating circulation desks, providing real-time Recharts visualization, and integrating conversational study assistants, the system eliminates manual administrative burdens, enforces stringent audit compliance, and elevates the academic learning experience for campus scholars.